

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
DEEP LEARNING

ĐỀ TÀI:

ỨNG DỤNG MẠNG NƠ-RON TÍCH CHẬP (CNN)
VÀ DEEPSORT TRONG HỆ THỐNG NHẬN DIỆN,
THEO DÕI VÀ CẢNH BÁO ĐEO KHẨU TRANG
THỜI GIAN THỰC

Giảng viên hướng dẫn: TS. Huỳnh Thái Học

TP. HỒ CHÍ MINH, NĂM 2026

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
DEEP LEARNING

ĐỀ TÀI:

ỨNG DỤNG MẠNG NƠ-RON TÍCH CHẬP (CNN)
VÀ DEEPSORT TRONG HỆ THỐNG NHẬN DIỆN,
THEO DÕI VÀ CẢNH BÁO ĐEO KHẨU TRANG
THỜI GIAN THỰC

Giảng viên hướng dẫn: TS. Huỳnh Thái Học
Sinh viên thực hiện: Phạm Nguyễn Hồng Ngọc – 2001230568
Trần Quốc Trường – 2001231017
Nguyễn Duy Hưng – 2001230327
Trần Thị Kim Ngân – 2001230554

TP. HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm 11 xin gửi lời cảm ơn chân thành đến Ban giám hiệu Trường Đại học Công thương TP.HCM cùng Khoa Công nghệ Thông tin đã tạo môi trường học tập tốt nhất cho chúng em trong suốt thời gian qua.

Đặc biệt, nhóm xin gửi lời tri ân sâu sắc đến TS. Huỳnh Thái Học – người thầy đã trực tiếp hướng dẫn, dành nhiều thời gian tâm huyết để dẫn dắt nhóm qua các buổi thực hành. Nhờ có sự định hướng tỉ mỉ của các thầy, nhóm đã tiếp cận môn học dưới góc nhìn toàn diện, nâng cao tư duy thực chiến và hiểu rõ hơn tính thiết thực của kiến thức vào thực tế.

Do giới hạn về mặt thời gian và trải nghiệm thực tế, bài báo cáo khó lòng hoàn hảo tuyệt đối. Nhóm 11 rất hy vọng sẽ nhận được sự lượng thứ cùng những nhận xét, đóng góp quý báu từ thầy để hoàn thiện năng lực của bản thân trong tương lai.

Kính chúc thầy luôn mạnh khỏe, hạnh phúc và thành công trên con đường giáo dục!

Nhóm 11 xin chân thành cảm ơn!

(Tập thể thành viên nhóm)

Phạm Nguyễn Hồng Ngọc – 2001230568

Trần Quốc Trường – 2001231017

Nguyễn Duy Hưng – 2001230327

Trần Thị Kim Ngân – 2001230554

MỤC LỤC

LỜI CẢM ƠN	2
DANH MỤC BẢNG BIỂU	5
DANH MỤC HÌNH ẢNH	6
LỜI MỞ ĐẦU	7
1 CHƯƠNG 1: MỞ ĐẦU	9
1.1 Lý do chọn đề tài	9
1.2 Mục tiêu nghiên cứu	9
1.2.1 Mục tiêu tổng quát	9
1.2.2 Mục tiêu cụ thể	9
2 CHƯƠNG 2: TỔNG QUAN CƠ SỞ LÝ THUYẾT	11
2.1 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)	11
2.1.1 Feature trong CNN	11
2.1.2 Những lớp cơ bản của mạng CNN	11
2.2 Thuật toán theo dõi đối tượng đa mục tiêu DeepSORT	12
2.2.1 Bài toán theo dõi đối tượng (Object Tracking)	12
2.2.2 Bộ lọc Kalman (Kalman Filter)	12
2.2.3 Thuật toán Hungary (Hungarian Algorithm)	13
2.2.4 Mạng nhúng sâu (Deep Appearance Descriptor)	13
3 CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG	14
3.1 Kiến trúc tổng quan	14
3.2 Xây dựng và tiền xử lý bộ dữ liệu (dataset)	15
3.2.1 Thu thập dữ liệu	15
3.2.2 Lọc ảnh chất lượng thấp	16
3.2.3 Gán nhãn dữ liệu	17
3.3 Quá trình huấn luyện mô hình CNN	18
3.3.1 Môi trường cấu hình	18
3.3.2 Kiến trúc mô hình MaskCNN	19
3.3.3 Tham số huấn luyện	21
3.4 Triển khai module theo dõi và cấu hình logic cảnh báo	22
3.4.1 Tích hợp CNN với DeepSORT	22
3.4.2 Xây dựng thuật toán đếm thời gian (Timer) theo ID đối tượng	22

3.4.3	Cơ chế kích hoạt cảnh báo	23
4	CHƯƠNG 4: CHƯƠNG TRÌNH THỰC NGHIỆM	25
4.1	Chỉ số đánh giá hiệu suất	25
4.2	Tiêu chí đánh giá mô hình	26
4.3	Kết quả thực nghiệm	27
4.3.1	So sánh hàm tối ưu SGD và Adam	27
4.3.2	Demo kết quả	31
5	CHƯƠNG 5: KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN	33
5.1	Kết quả huấn luyện mô hình CNN	33
5.1.1	Ưu điểm	33
5.1.2	Nhược điểm	33
5.2	Kết quả thử nghiệm thời gian thực trên Webcam	33
5.3	Kết luận	34
5.4	Hướng phát triển của đề tài	34
	TÀI LIỆU THAM KHẢO	36

DANH MỤC BẢNG BIỂU

3.1	Các giai đoạn xử lý tuần tự trong kiến trúc pipeline của hệ thống	14
3.2	Các lớp dữ liệu trong bộ Face Mask Dataset	16
3.3	Thống kê số lượng ảnh theo lớp và tập dữ liệu	17
3.4	Các kỹ thuật tăng cường dữ liệu áp dụng cho tập huấn luyện	18
3.5	Cấu hình phần cứng và phần mềm môi trường huấn luyện	19
3.6	Cấu trúc chi tiết mô hình MaskCNN	20
3.7	Cấu hình siêu tham số (Hyperparameters) trong quá trình huấn luyện . .	21
3.8	Nguyên tắc hoạt động và xử lý logic của bộ đếm thời gian (Timer)	23
3.9	Các thành phần hiển thị trực quan và cấu hình màu sắc trên giao diện cảnh báo	24

DANH MỤC HÌNH ẢNH

3.1	Sơ đồ luồng xử lý tổng quan của hệ thống	15
3.2	Phân bố dữ liệu của bộ Face Mask Dataset sau khi chia tập Train, Validation và Test	18
3.3	Trực quan hóa Feature Map tại các khối tích chập của mô hình MaskCNN	21
4.1	Đồ thị Loss và Accuracy của mô hình sử dụng bộ tối ưu SGD.	27
4.2	Báo cáo hiệu suất phân loại (Classification Report) với SGD.	28
4.3	Ma trận nhầm lẫn của mô hình sử dụng bộ tối ưu SGD.	28
4.4	Đồ thị Loss và Accuracy của mô hình sử dụng bộ tối ưu Adam.	29
4.5	Báo cáo hiệu suất phân loại (Classification Report) với Adam.	29
4.6	Ma trận nhầm lẫn của mô hình sử dụng bộ tối ưu Adam.	30
4.7	So sánh kết quả nhận diện trên ảnh tĩnh giữa mô hình sử dụng Adam và SGD.	31
4.8	Demo kết quả nhận diện và theo dõi trên webcam thời gian thực.	32

LỜI MỞ ĐẦU

Trong những năm gần đây, Trí tuệ nhân tạo (AI) đã và đang khẳng định vai trò cốt lõi trong cuộc cách mạng công nghệ toàn cầu. Sự bùng nổ về năng lực tính toán của phần cứng (GPU) cùng với lượng dữ liệu số khổng lồ được thu thập đã đưa Học máy (Machine Learning) tiến một bước dài, tạo tiền đề cho sự ra đời của Học sâu (Deep Learning). Các mô hình Học sâu đã giúp máy tính thực thi được những tác vụ vô cùng phức tạp với độ chính xác vượt trội, từ phân loại hàng ngàn vật thể, xử lý ngôn ngữ tự nhiên cho đến các hệ thống tự hành. Nhờ đó, việc ứng dụng Học sâu vào các hệ thống camera giám sát thông minh (Smart Surveillance) đã trở thành một xu hướng tất yếu trong kỷ nguyên số.

Trên thực tế, công tác đảm bảo an ninh, trật tự tại các khu vực đặc thù và có tính chất rủi ro cao như hệ thống ngân hàng, phòng giao dịch hay các điểm rút tiền tự động (ATM) luôn là bài toán thách thức đối với các nhà quản lý. Đây là những nơi thường xuyên diễn ra các giao dịch tài chính lớn, dễ trở thành mục tiêu hàng đầu của tội phạm cướp giật, trộm cắp tài sản. Thủ đoạn phổ biến của các đối tượng này là cố tình che giấu nhân dạng bằng cách đeo khẩu trang, đội mũ kín hoặc che mặt trước khi bước vào khu vực giao dịch nhằm vô hiệu hóa khả năng nhận diện của hệ thống camera truyền thống. Mặc dù các camera hiện nay có mật độ dày đặc, nhưng chúng chỉ hoạt động theo cơ chế ghi hình thụ động và phụ thuộc hoàn toàn vào sức người để theo dõi màn hình, dẫn đến việc khó phát hiện và ngăn chặn kịp thời các hành vi nghi vấn diễn ra trong thời gian ngắn.

Để giải quyết triệt để hạn chế trên, việc xây dựng một hệ thống giám sát chủ động, có khả năng tự động phân tích hành vi che mặt và đưa ra cảnh báo tức thời là vô cùng cấp thiết. Xuất phát từ nhu cầu thực tiễn đó, nhóm 11 đã tập trung nghiên cứu, phát triển đề tài "Ứng dụng mạng nơ-ron tích chập (CNN) và DeepSORT trong hệ thống nhận diện, theo dõi và cảnh báo đeo khẩu trang thời gian thực". Điểm đột phá của đề tài là việc kết hợp mô hình thị giác máy tính dựa trên mạng nơ-ron tích chập (CNN) để phát hiện trạng thái khuôn mặt, kết hợp với thuật toán theo dõi đa mục tiêu DeepSORT nhằm duy trì định danh, định vị quỹ đạo và thiết lập logic đếm thời gian. Hệ thống sẽ tự động kích hoạt tín hiệu cảnh báo đến bộ phận an ninh ngay khi phát hiện có đối tượng cố tình che mặt liên tục vượt quá khung thời gian an toàn quy định (mặc định 10 giây, có thể cấu hình linh hoạt).

Nội dung chính của báo cáo đồ án được kết cấu thành 5 chương như sau:

- **Chương 1: Mở đầu** – Trình bày lý do chọn đề tài, mục tiêu nghiên cứu tổng quát và cụ thể, đồng thời định hướng nội dung và phạm vi nghiên cứu của đề tài.

- **Chương 2: Tổng quan cơ sở lý thuyết** – Trình bày nền tảng khoa học về mạng nơ-ron tích chập (CNN) trong bài toán phát hiện vật thể, nguyên lý hoạt động của thuật toán theo dõi đối tượng DeepSORT và cơ chế lọc Kalman.
- **Chương 3: Thiết kế và xây dựng** – Trình bày kiến trúc tổng quan của hệ thống, quá trình xây dựng và tiền xử lý bộ dữ liệu, thiết kế mô hình CNN, quá trình huấn luyện, tích hợp thuật toán DeepSORT và xây dựng cơ chế theo dõi, cảnh báo theo thời gian thực.
- **Chương 4: Chương trình thực nghiệm** – Mô tả các chỉ số và tiêu chí đánh giá mô hình, trình bày kết quả thực nghiệm, so sánh hiệu năng giữa các thuật toán tối ưu hóa (SGD và Adam), đồng thời minh họa kết quả hoạt động của hệ thống trên ảnh và webcam.
- **Chương 5: Kết quả và phương hướng phát triển** – Tổng kết các kết quả đạt được của mô hình và hệ thống, phân tích ưu điểm, hạn chế còn tồn tại, đồng thời đề xuất các hướng phát triển và cải tiến trong tương lai nhằm nâng cao hiệu quả ứng dụng thực tế.

1 CHƯƠNG 1: MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong bối cảnh an ninh tại các ngân hàng và trung tâm tài chính ngày càng trở nên phức tạp, việc phát hiện và ngăn chặn kịp thời các đối tượng cố tình che giấu nhân dạng (đeo khẩu trang, kính râm, mũ kín) là rất cần thiết. Tuy nhiên, các hệ thống camera giám sát hiện nay chủ yếu chỉ ghi hình thụ động, thiếu khả năng phân tích và cảnh báo chủ động thời gian thực. Nhân viên an ninh khó có thể theo dõi liên tục nhiều màn hình cùng lúc, dẫn đến bỏ sót các hành vi đáng ngờ.

Với sự phát triển mạnh mẽ của Trí tuệ nhân tạo, đặc biệt là Thị giác máy tính, việc áp dụng Mạng nơ-ron tích chập (CNN) để nhận diện trạng thái đeo khẩu trang và DeepSORT để theo dõi, định danh và tính toán thời gian che mặt của đối tượng đã trở thành giải pháp hiệu quả. Khi thời gian che mặt vượt quá ngưỡng cho phép (mặc định 10 giây, có thể cấu hình linh hoạt), hệ thống sẽ tự động kích hoạt cảnh báo giúp lực lượng an ninh kịp thời can thiệp.

Xuất phát từ nhu cầu thực tiễn nâng cao năng lực giám sát an ninh chủ động tại các tổ chức tín dụng, nhóm quyết định thực hiện đề tài: "Ứng dụng Mạng nơ-ron tích chập (CNN) và DeepSORT trong hệ thống nhận diện, theo dõi và cảnh báo đeo khẩu trang thời gian thực".

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và xây dựng thành công một hệ thống giám sát an ninh thông minh có khả năng tự động nhận diện hành vi đeo khẩu trang/che mặt, theo dõi quỹ đạo di chuyển của đối tượng trong thời gian thực và tự động phát tín hiệu cảnh báo đến bộ phận quản lý khi đối tượng vi phạm quy định về thời gian an toàn tại các khu vực trọng yếu như ngân hàng.

1.2.2 Mục tiêu cụ thể

- **Về mặt lý thuyết:** Nghiên cứu và nắm vững nguyên lý hoạt động của các kiến trúc mạng nơ-ron tích chập (CNN) trong bài toán phát hiện vật thể (Object Detection); làm chủ thuật toán theo dõi đối tượng DeepSORT ứng dụng bộ lọc Kalman và mạng nhúng sâu (Deep Appearance Descriptor).
- **Về mặt dữ liệu:** Thu thập, chuẩn hóa và gán nhãn bộ dữ liệu hình ảnh (Dataset) chất lượng cao bao gồm các trạng thái: đeo khẩu trang, không đeo khẩu trang, và các hành vi che mặt khác phù hợp với bối cảnh camera giám sát góc cao tại ngân hàng.

- **Về mặt công nghệ:** Huấn luyện thành công mô hình CNN đạt độ chính xác cao ($Accuracy \geq 90\%$), tốc độ xử lý nhanh (FPS đáp ứng thời gian thực) trên các luồng video livestream trực tiếp. Tích hợp module DeepSORT để duy trì ID nhân dạng ổn định, không bị mất dấu ngay cả khi đối tượng bị che khuất tạm thời.
- **Về mặt tính năng cảnh báo:** Phát triển thuật toán logic đếm thời gian (Timer) dựa trên ID của DeepSORT. Thiết lập hệ thống kích hoạt âm thanh/cảnh báo hiển thị trực quan khi một ID duy trì trạng thái che mặt liên tục quá thời gian cấu hình (mặc định 10 giây).
- **Về mặt ứng dụng và thực nghiệm:** Đánh giá hiệu năng của mô hình thông qua môi trường mô phỏng bằng webcam thời gian thực; bước đầu phân tích, đo lường các chỉ số sai số (như tỷ lệ nhận diện sai, tỷ lệ báo động giả - False Alarm Rate) trong không gian phòng thí nghiệm, từ đó đề xuất định hướng tối ưu phần cứng và giải pháp triển khai hệ thống thực tế trong tương lai.

2 CHƯƠNG 2: TỔNG QUAN CƠ SỞ LÝ THUYẾT

2.1 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)

Mạng nơ-ron tích chập (CNN) là một mô hình học sâu (*deep learning*) được sử dụng rộng rãi để xử lý dữ liệu dạng hình ảnh, dựa trên cơ chế tổ chức sắp xếp của vỏ não thị giác ở động vật [1]. Mục tiêu thiết kế cốt lõi của kiến trúc này là tự động học hỏi một cách linh hoạt các cấu trúc phân cấp không gian của các đặc trưng, từ các dạng mẫu cấp thấp đến cấp cao. Mô hình này đã chứng minh được hiệu năng vượt trội trong nhiều tác vụ khác nhau, bao gồm nhận diện khuôn mặt, định danh vật thể và phát hiện biến báo giao thông – đặc biệt là trong lĩnh vực robot và xe tự hành. Việc giảm thiểu số lượng tham số so với mạng nơ-ron nhân tạo (ANN) truyền thống là một trong những đặc điểm quan trọng nhất của CNN [1].

Mục đích chính của CNN là trích xuất và học các đặc trưng có giá trị từ dữ liệu đầu vào. Trong quy trình này, các lớp khởi đầu hoạt động như một tập hợp các bộ trích xuất đặc trưng tích chập (*convolutional feature extractors*) dựa trên các bộ lọc (*filters*) có khả năng tự học hỏi thông số. Các bộ lọc này hoạt động như một cửa sổ trượt (*sliding window*) di chuyển liên tục trên từng vùng của dữ liệu hình ảnh đầu vào. Trong đó, khoảng cách dịch chuyển chồng lấp giữa các bước trượt được gọi là bước nhảy (*stride*), và kết quả đầu ra thu được gọi là các bản đồ đặc trưng (*feature maps*).

Các lõi tích chập (*convolutional kernels*) cấu thành nên mỗi lớp trong mạng CNN và được sử dụng để tạo ra các bản đồ đặc trưng khác nhau. Các vùng nơ-ron lân cận sẽ được kết nối với một nơ-ron cụ thể thuộc bản đồ đặc trưng của lớp kế tiếp. Để tạo ra một bản đồ đặc trưng, lõi tích chập phải được chia sẻ trọng số trên toàn bộ các vị trí không gian của đầu vào. Sau khi các lớp tích chập (*convolutional layers*) và lớp nén/gộp (*pooling layers*) được thiết lập, một hoặc nhiều lớp kết nối đầy đủ (*fully connected layers*) sẽ được sử dụng ở giai đoạn cuối để hoàn thiện tác vụ phân loại [1].

2.1.1 Feature trong CNN

Các feature (đặc trưng) được sử dụng để trích xuất thông tin từ ảnh đầu vào. Các feature này được học tự động từ dữ liệu và được sử dụng để giảm thiểu số lượng thông tin cần xử lý trong quá trình huấn luyện mạng. Các feature này có thể là các đường cạnh, các vùng sáng tối, các đường cong và các đối tượng phức tạp hơn. Các feature này được trích xuất thông qua các bộ lọc tích chập và được kết hợp lại để tạo thành các feature maps, đóng vai trò quan trọng trong việc phân loại ảnh.

2.1.2 Những lớp cơ bản của mạng CNN

Lớp tích chập (Convolutional Layer): Đây là thành phần quan trọng nhất của toàn bộ mạng CNN. Các yếu tố cốt lõi trong lớp này bao gồm: filter (bộ lọc), stride (bước dịch),

padding (phần đệm) và feature map (bản đồ đặc trưng).

- **Filter (Kernel):** Mạng CNN sử dụng các filter để áp dụng vào từng vùng của ma trận hình ảnh. Các filter thường là một ma trận trọng số hai chiều (2D) giúp phát hiện các đặc trưng trong hình ảnh.[2]
- **Stride:** Là bước di chuyển của kernel trên ma trận đầu vào, xác định khoảng cách giữa các lần quét. Stride lớn sẽ tạo ra đầu ra nhỏ hơn.[2]
- **Padding:** Được sử dụng khi kích thước của filter không khớp với kích thước của hình ảnh đầu vào.[2]
- **Feature map:** Ma trận kết quả thu được sau khi filter quét qua toàn bộ ma trận ảnh đầu vào. Đóng vai trò lưu giữ, trích xuất và làm nổi bật các đặc điểm quan trọng (như đường nét, màu sắc, hình khối) từ dữ liệu đầu vào.

Activation Layer: Sau khi thực hiện phép tích chập, dữ liệu sẽ đi qua lớp kích hoạt để thêm tính phi tuyến vào mô hình. Hàm kích hoạt phổ biến nhất là ReLU (Rectified Linear Unit).

Pooling Layer: Hay còn gọi là downsampling, có nhiệm vụ giảm chiều dữ liệu và giảm số lượng tham số trong đầu vào.[2]

Fully-connected Layer: Mỗi đầu vào được liên kết với đầu ra thông qua các trọng số có thể học được. Thông thường, lớp fully connected cuối cùng sẽ được theo sau bởi một bộ phân loại Softmax.[1]

2.2 Thuật toán theo dõi đối tượng đa mục tiêu DeepSORT

2.2.1 Bài toán theo dõi đối tượng (Object Tracking)

Bài toán theo dõi đối tượng đa mục tiêu (Multiple Object Tracking - MOT) là nhiệm vụ phát hiện nhiều đối tượng trong video hoặc hình ảnh, dự đoán quỹ đạo chuyển động và duy trì danh tính (ID) nhất quán qua các khung hình, ngay cả khi chúng có thể bị che khuất hoặc tạm thời biến mất khỏi tầm nhìn. Để đạt được điều này, các thuật toán theo dõi đa đối tượng thường sử dụng sự kết hợp của các kỹ thuật phát hiện đối tượng, liên kết dữ liệu và dự đoán quỹ đạo. Các thuật toán này cũng phải giải quyết các thách thức như xử lý sự che khuất, duy trì nhận dạng đối tượng trong các cảnh đông đúc và thích ứng với những thay đổi về hình dạng hoặc chuyển động của đối tượng.

2.2.2 Bộ lọc Kalman (Kalman Filter)

Bộ lọc Kalman là thành phần cốt lõi để mô hình hóa chuyển động của đối tượng trong không gian hai chiều. Nó dự đoán vị trí và vận tốc của đối tượng ở khung hình tiếp theo

dựa trên trạng thái trước đó. Trong DeepSORT, không gian trạng thái thường là một vector cấu trúc 8 chiều:

$$x = (u, v, a, h, u', v', a', h')$$

Trong đó (u, v) là tọa độ trung tâm, a là tỷ lệ khung hình, h là chiều cao, và các ký hiệu có dấu phẩy (u', v', a', h') biểu thị các vận tốc tương ứng. Bộ lọc Kalman thực hiện hai bước chính là **Predict** (Dự đoán vị trí mới) và **Update** (Cập nhật dữ liệu khi thu được đo lường thực tế từ module detection). Nó giúp duy trì quỹ đạo ổn định và xử lý hiệu quả các trường hợp đối tượng tạm thời bị che khuất ngắn hạn.

2.2.3 Thuật toán Hungary (Hungarian Algorithm)

Thuật toán Hungarian dùng để giải bài toán gán tối ưu. Nó ghép nối các detection mới với các track đang tồn tại sao cho tổng chi phí (được tính bằng khoảng cách Mahalanobis và khoảng cách Cosine) là nhỏ nhất.

2.2.4 Mạng nhúng sâu (Deep Appearance Descriptor)

Deep Appearance Descriptor là một mạng CNN được huấn luyện để trích xuất vector embedding mô tả ngoại hình của đối tượng nhằm hỗ trợ nhận diện lại đối tượng khi bị che khuất ngắn hạn.

3 CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG

3.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo kiến trúc pipeline đa tầng, kết hợp các kỹ thuật học sâu (*deep learning*) với thuật toán theo dõi đối tượng thời gian thực. Luồng xử lý chính bao gồm năm giai đoạn tuần tự được mô tả chi tiết trong Bảng 3.1.

Bảng 3.1: Các giai đoạn xử lý tuần tự trong kiến trúc pipeline của hệ thống

Giai đoạn	Module	Chức năng
1	Video đầu vào	Nhận luồng video từ webcam, file video (.mp4) hoặc ảnh tĩnh thông qua OpenCV (<code>cv2.VideoCapture</code>).
2	MTCNN	Phát hiện và định vị khuôn mặt trong từng frame, lọc theo ngưỡng confidence ≥ 0.9 , trả về bounding box (x_1, y_1, x_2, y_2) .
3	MaskCNN	Phân loại khuôn mặt vừa phát hiện thành một trong ba trạng thái: Đeo đúng (<code>with_mask</code>), Đeo sai (<code>mask_wearred_incorrect</code>), Không đeo (<code>without_mask</code>).
4	DeepSORT	Gán và duy trì Track ID nhất quán cho từng người qua nhiều frame liên tục, sử dụng phương pháp kết hợp IoU và <i>appearance embedding</i> .
5	Timer & Alert	Đếm tích lũy thời gian đeo khẩu trang đúng cách theo từng Track ID; kích hoạt cảnh báo trực quan khi vượt ngưỡng 10 giây.

Khi phát hiện đối tượng vi phạm (đeo khẩu trang đúng cách liên tục vượt quá 10 giây), hệ thống hiển thị banner cảnh báo màu đỏ trực tiếp lên frame video với thông tin ID người dùng và thời gian vi phạm. Sơ đồ luồng xử lý chi tiết được thể hiện trong Hình 3.1.



Hình 3.1: Sơ đồ luồng xử lý tổng quan của hệ thống

3.2 Xây dựng và tiền xử lý bộ dữ liệu (dataset)

3.2.1 Thu thập dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là *Face Mask Dataset* (FMD_DATASET) được công bố trên nền tảng Kaggle bởi tác giả Shiekh Burhan. Bộ dữ liệu bao gồm các ảnh khuôn mặt trong nhiều điều kiện khác nhau nhằm phục vụ cho bài toán phân loại trạng thái đeo khẩu trang.

Dataset được tải trực tiếp trong môi trường Google Colaboratory thông qua Kaggle API bằng lệnh:

```
!kaggle datasets download \
-d shiekhburhan/face-mask-dataset \
-p /content/dataset --unzip
```

Sau khi giải nén, dữ liệu được tổ chức theo cấu trúc thư mục chuẩn của lớp ImageFolder

trong thư viện PyTorch. Mỗi thư mục con tương ứng với một lớp dữ liệu riêng biệt. Chi tiết được trình bày trong Bảng 3.2.

Bảng 3.2: Các lớp dữ liệu trong bộ Face Mask Dataset

Tên thư mục	Nhãn	Mô tả
incorrect_mask	Class 0	Người đeo khẩu trang không đúng cách như hở mũi, hở cằm hoặc đeo lệch.
with_mask	Class 1	Người đeo khẩu trang đúng quy cách, che kín mũi và miệng.
without_mask	Class 2	Người không đeo khẩu trang.

3.2.2 Lọc ảnh chất lượng thấp

Trước khi tiến hành chia tập dữ liệu, một bước tiền xử lý quan trọng được thực hiện là loại bỏ toàn bộ các ảnh có kích thước nhỏ hơn 64×64 pixels. Các ảnh có độ phân giải thấp thường chứa rất ít thông tin đặc trưng hữu ích về khuôn mặt, gây nhiễu và làm giảm chất lượng quá trình học của mô hình. Việc loại bỏ các ảnh này giúp cải thiện đáng kể chất lượng tập dữ liệu và khả năng tổng quát hóa của mô hình.

Quá trình lọc được thực hiện bằng đoạn mã Python sử dụng thư viện PIL và pathlib:

```
for img_path in Path(DATASET_ROOT).rglob("*"):
    if img_path.suffix.lower() not in \
        [".jpg", ".jpeg", ".png", ".bmp", ".webp"]:
        continue
    with Image.open(img_path) as img:
        w, h = img.size
        if min(w, h) < 64:
            os.remove(img_path)
```

Sau bước lọc này, toàn bộ ảnh còn lại đều đảm bảo cạnh ngắn nhất tối thiểu là 64 pixels, đảm bảo chất lượng đầu vào đồng nhất cho quá trình huấn luyện.

Toàn bộ dữ liệu được chia ngẫu nhiên theo tỷ lệ 70%-15%-15% tương ứng cho ba tập dữ liệu:

- **Training Set (70%):** dùng để huấn luyện và cập nhật trọng số của mô hình.
- **Validation Set (15%):** dùng để đánh giá mô hình trong quá trình huấn luyện và hỗ trợ lựa chọn siêu tham số.
- **Test Set (15%):** dùng để đánh giá hiệu năng cuối cùng của mô hình trên dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

Thống kê phân phối dữ liệu thực tế sau khi chia tập được trình bày trong Bảng 3.3.

Bảng 3.3: Thống kê số lượng ảnh theo lớp và tập dữ liệu

Lớp	Train	Validation	Test	Tổng
Incorrect Mask (Class 0)	3.200	723	752	4.675
With Mask (Class 1)	3.200	700	695	4.595
Without Mask (Class 2)	3.200	732	709	4.641
Tổng	9.600	2.155	2.156	14.536

Để hạn chế hiện tượng mất cân bằng dữ liệu giữa các lớp, hệ thống sử dụng hàm `limit_dataset_per_class()` nhằm giới hạn số lượng ảnh tối đa của mỗi lớp. Trong quá trình thực nghiệm, giá trị `max_per_class` được thiết lập bằng 3.200. Nếu số lượng ảnh của một lớp vượt quá ngưỡng này, các mẫu sẽ được lựa chọn ngẫu nhiên (*Random Sampling*) với seed cố định (`random.seed(42)`) nhằm đảm bảo tính tái lập và cân bằng tương đối giữa các lớp dữ liệu.

3.2.3 Gán nhãn dữ liệu

Bộ dữ liệu `FMD_DATASET` đã được gán nhãn sẵn theo cấu trúc thư mục. Thư viện PyTorch tự động ánh xạ tên thư mục sang nhãn số nguyên thông qua thuộc tính `class_to_idx` của lớp `ImageFolder` như sau:

```
{
    'incorrect_mask': 0,
    'with_mask': 1,
    'without_mask': 2
}
```

Do đó, quá trình gán nhãn được thực hiện hoàn toàn tự động và không cần can thiệp thủ công.

Các kỹ thuật tăng cường dữ liệu (*Data Augmentation*) được áp dụng riêng cho tập huấn luyện nhằm tăng tính đa dạng của dữ liệu và cải thiện khả năng tổng quát hóa của mô hình. Các cấu hình được trình bày trong Bảng 3.4.

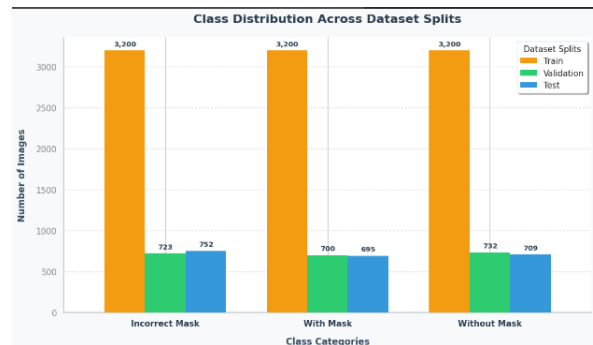
Bảng 3.4: Các kỹ thuật tăng cường dữ liệu áp dụng cho tập huấn luyện

Kỹ thuật	Tham số	Mục đích
Resize	512×512	Chuẩn hóa kích thước đầu vào của ảnh.
RandomHorizontalFlip	$p = 0.5$	Tạo thêm các biến thể đối xứng của khuôn mặt.
RandomRotation	$\pm 15^\circ$	Mô phỏng các góc quay khác nhau của khuôn mặt.
ColorJitter	Brightness = 0.3 Contrast = 0.3 Saturation = 0.3 Hue = 0.1	Mô phỏng các điều kiện ánh sáng khác nhau trong thực tế(camera giám sát góc cao, ngược sáng, ánh đèn huỳnh quang), chống hiện tượng rò rỉ đặc trưng ánh sáng (<i>lighting leakage</i>).
Normalize	Mean = [0.485, 0.456, 0.406] Std = [0.229, 0.224, 0.225]	Chuẩn hóa dữ liệu theo thống kê ImageNet giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn.

Đối với tập *Validation* và *Test*, chỉ hai phép biến đổi cơ bản là Resize (512×512) và Normalize được áp dụng. Các kỹ thuật tăng cường dữ liệu như xoay ảnh, lật ảnh hoặc thay đổi độ sáng không được sử dụng nhằm đảm bảo tính khách quan trong quá trình đánh giá hiệu năng của mô hình.

FACE MASK DETECTION DATASET SUMMARY			
▶ TOTAL IMAGES :	13,911		
▶ TRAINING SET :	9,600	(~69.0%)	
▶ VAL SET :	2,155	(~15.5%)	
▶ TEST SET :	2,156	(~15.5%)	
CLASS DISTRIBUTION DETAILS:			
	Train	Validation	Test
Incorrect Mask	3200	723	752
With Mask	3200	700	695
Without Mask	3200	732	709

(a) Thống kê số lượng ảnh của từng lớp



(b) Biểu đồ phân bố dữ liệu

Hình 3.2: Phân bố dữ liệu của bộ Face Mask Dataset sau khi chia tập Train, Validation và Test

3.3 Quá trình huấn luyện mô hình CNN

3.3.1 Môi trường cấu hình

Quá trình huấn luyện được thực hiện trên nền tảng Google Colaboratory (Google Colab) với GPU Tesla T4 do Google cung cấp miễn phí. Tập dữ liệu được lưu trữ trên Google Drive và mount vào Colab runtime thông qua API:

```
from google.colab import drive
drive.mount('/content/drive')
```

Cấu hình phần cứng và phần mềm môi trường huấn luyện được thống kê chi tiết trong Bảng 3.5:

Bảng 3.5: Cấu hình phần cứng và phần mềm môi trường huấn luyện

Thành phần	Chi tiết
Nền tảng	Google Colaboratory (Colab).
GPU	NVIDIA Tesla T4 (15 GB VRAM).
CPU	Intel Xeon (2 vCPU).
RAM	12.7 GB.
Ngôn ngữ lập trình	Python 3.10+.
Deep Learning Framework	PyTorch 2.4+ / TorchVision 0.19+.
Face Detection	facenet-pytorch (MTCNN, InceptionResnetV1).
Object Tracking	deep-sort-realtime.
Xử lý ảnh / Video	OpenCV (cv2), Pillow (PIL).
Khoa học dữ liệu	NumPy 2.x, Matplotlib, Seaborn, scikit-learn.

Toàn bộ thư viện được cài đặt bằng pip tại thời điểm khởi động runtime:

```
!pip install torch torchvision opencv-python pillow
!pip install deep-sort-realtime facenet-pytorch soundfile
```

3.3.2 Kiến trúc mô hình MaskCNN

Mô hình MaskCNN được xây dựng hoàn toàn bằng thư viện PyTorch nhằm thực hiện bài toán phân loại trạng thái đeo khẩu trang gồm ba lớp: *incorrect_mask*, *with_mask* và *without_mask*. Kiến trúc mạng bao gồm bốn khối tích chập liên tiếp (*Convolution Block*), một tầng *Adaptive Global Average Pooling* và ba lớp kết nối đầy đủ (*Fully Connected Layer*).

Mỗi khối tích chập được cấu thành từ các thành phần: lớp tích chập (*Convolution*), chuẩn hóa theo lô (*Batch Normalization*), hàm kích hoạt *ReLU*, lớp giảm kích thước đặc trưng *MaxPooling* và lớp *Dropout* nhằm hạn chế hiện tượng quá khớp (*Overfitting*). Mỗi lớp kết nối đầy đủ cũng được trang bị thêm *Batch Normalization* và *Dropout* để ổn định quá trình học.

Chi tiết kiến trúc mô hình được trình bày trong Bảng 3.6.

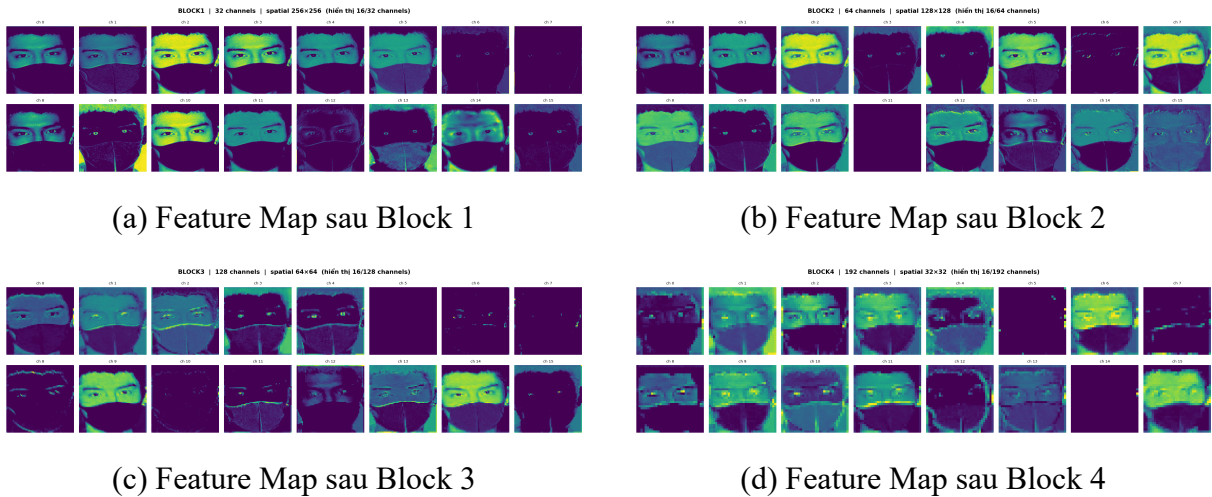
Bảng 3.6: Cấu trúc chi tiết mô hình MaskCNN

Tầng	Thao tác	Kích thước đầu ra	Số bộ lọc
Input	Ảnh RGB đầu vào	$512 \times 512 \times 3$	3 kênh
Conv Block 1	Conv(3→32, k=3, pad=1) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$256 \times 256 \times 32$	32
Conv Block 2	Conv(32→64, k=3, pad=1) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$128 \times 128 \times 64$	64
Conv Block 3	Conv(64→128, k=3, pad=1) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$64 \times 64 \times 128$	128
Conv Block 4	Conv(128→192, k=3, pad=1) + BN + ReLU + MaxPool(2) + Dropout(0.30)	$32 \times 32 \times 192$	192
Adaptive Avg Pooling	AdaptiveAvgPool2d(4, 4)	$4 \times 4 \times 192$	—
Fully Connected 1	Flatten + Linear(3072→128) + BN1d + ReLU + Dropout(0.5)	128	128 neurons
Fully Connected 2	Linear(128→64) + BN1d + ReLU + Dropout(0.4)	64	64 neurons
Output Layer	Linear(64→3)	3	3 lớp

Với ảnh đầu vào kích thước 512×512 pixels, các phép MaxPooling làm giảm dần kích thước không gian của bản đồ đặc trưng (Feature Map) theo chuỗi:

$$512 \times 512 \rightarrow 256 \times 256 \rightarrow 128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32$$

Trong khi đó số lượng kênh đặc trưng tăng dần từ 32 lên 192, giúp mô hình học được các đặc trưng ngày càng trừu tượng và có tính phân biệt cao hơn. Sau khối tích chập cuối cùng, tầng Adaptive Average Pooling chuẩn hóa kích thước đầu ra về $4 \times 4 \times 192$ (tổng 3.072 giá trị) trước khi đưa vào các lớp Fully Connected để thực hiện phân loại.



Hình 3.3: Trực quan hóa Feature Map tại các khối tích chập của mô hình MaskCNN

3.3.3 Tham số huấn luyện

Các tham số và cấu hình cụ thể áp dụng cho quá trình huấn luyện mô hình được thống kê chi tiết trong Bảng 3.7.

Bảng 3.7: Cấu hình siêu tham số (Hyperparameters) trong quá trình huấn luyện

Tham số	Giá trị / Cấu hình
Số epochs tối đa	50 (có Early Stopping, patience=5)
Batch size	32
Optimizer (chính)	Adam (lr=0.001, weight_decay=5e-4)
Optimizer (so sánh)	SGD (lr=0.01, momentum=0.9, weight_decay=1e-4)
Loss function	CrossEntropyLoss với label_smoothing=0.1
Learning Rate Scheduler	ReduceLROnPlateau (factor=0.5, patience=2, mode='min')
DataLoader workers	2 (num_workers=2)
Max ảnh mỗi lớp	3.200 ảnh (cân bằng dataset)
Lọc ảnh đầu vào	Loại bỏ ảnh có cạnh ngắn < 64 px

Chiến lược *Early Stopping* được áp dụng để ngăn chặn hiện tượng quá khớp (*overfitting*): nếu giá trị lỗi trên tập kiểm định (*val_loss*) không giảm sau 5 epoch liên tiếp, quá trình huấn luyện sẽ dừng sớm và mô hình có kết quả tốt nhất đã lưu sẽ được sử dụng. Mô hình tối ưu nhất (*best_val_loss* thấp nhất) được lưu tự động vào Google Drive thông qua dòng lệnh:

```
torch.save(model.state_dict(),
            '/content/drive/MyDrive/best_model_adam.pth')
```

Hàm `train_model()` thực hiện ghi lại toàn bộ lịch sử huấn luyện bao gồm các chỉ số `train_loss`, `val_loss`, `train_acc`, và `val_acc` theo từng epoch, sau đó xuất ra file định dạng JSON để phục vụ cho các tác vụ phân tích và trực quan hóa dữ liệu về sau.

3.4 Triển khai module theo dõi và cấu hình logic cảnh báo

3.4.1 Tích hợp CNN với DeepSORT

Class `PersonTrackerV2` là thành phần trung tâm của hệ thống, đảm nhiệm việc kết nối đầu ra phân loại của MaskCNN với thuật toán theo dõi DeepSORT. Quy trình tích hợp diễn ra trong hàm `process_video_v2()` theo các bước tuần tự sau:

- **Bước 1 — Phát hiện khuôn mặt:** MTCNN quét frame và trả về danh sách *bounding box* cùng điểm *confidence*. Chỉ các hộp có *confidence* ≥ 0.9 mới được giữ lại.
- **Bước 2 — Phân loại:** Vùng ảnh khuôn mặt được cắt ra, thực hiện thay đổi kích thước (*resize*) về 512×512 , chuẩn hóa bằng các thông số ImageNet (*mean/std*) rồi đưa qua mô hình MaskCNN. Hàm `argmax()` được sử dụng để chọn ra lớp (*class*) có xác suất cao nhất.
- **Bước 3 — Chuẩn hóa định dạng:** Bounding box từ định dạng ban đầu *xyxy* được chuyển sang định dạng *xywh* theo yêu cầu đầu vào của hàm `DeepSort.update_tracks()`, cụ thể là $[x_1, y_1, w, h]$.
- **Bước 4 — DeepSORT update:** Hàm `tracker.update_tracks(ds_input, frame=frame)` thực hiện thuật toán Hungarian (*Hungarian Algorithm*) để khớp kết quả phát hiện (*detection*) với các vết theo dõi (*track*) hiện có, đồng thời tạo *track* mới hoặc xóa bỏ các *track* cũ. Chỉ các *track* ở trạng thái *Confirmed* (duy trì liên tục trong `n_init=3` frame) mới được đưa vào xử lý ở các bước tiếp theo.

Cấu hình khởi tạo của `DeepSort` trong mã nguồn được thiết lập cụ thể như sau:

```
self.tracker = DeepSort(  
    max_age=30,          # Giữ track tối đa 30 frame khi mất detect  
    n_init=3,            # Cần 3 frame để xác nhận track mới  
    nn_budget=100,       # Số embedding tối đa lưu trong gallery  
    max_iou_distance=0.7 # Ngưỡng IoU để khớp detection-track  
)
```

3.4.2 Xây dựng thuật toán đếm thời gian (Timer) theo ID đối tượng

Thuật toán đếm thời gian được thiết kế để theo dõi tích lũy thời gian đeo khẩu trang đúng cách cho từng cá nhân dựa trên `Track ID`. Mỗi `track_id` được quản lý bởi một dictionary trạng thái độc lập:

```

self.states = defaultdict(lambda: {
    "mask_start":    None,    # Thời điểm bắt đầu đeo đúng
    "mask_duration": 0.0,    # Tổng thời gian tích lũy (giây)
    "last_class":    None,    # Trạng thái lần cuối
    "alerted":       False,   # Đã cảnh báo chưa
})

```

Logic đếm thời gian hoạt động theo nguyên tắc được thống kê chi tiết trong Bảng 3.8.

Bảng 3.8: Nguyên tắc hoạt động và xử lý logic của bộ đếm thời gian (Timer)

Trạng thái phát hiện (class_id)	Hành động của Timer
class_id == 1 (đeo đúng) và mask_start là None	Ghi nhận thời điểm bắt đầu: mask_start = current_time.
class_id == 1 (đeo đúng) và mask_start đã có	Cập nhật thời gian: mask_duration = current_time - mask_start.
class_id != 1 (đeo sai hoặc không đeo)	Reset hoàn toàn: mask_start = None, mask_duration = 0.0, alerted = False.
Track ID mất khỏi frame (max_age hết)	DeepSORT tự xóa track; states[tid] không bị xóa ngay (dùng defaultdict).

Đây là thiết kế quan trọng: đồng hồ chỉ chạy khi đối tượng đeo khẩu trang đúng cách liên tục (class_id == 1), và sẽ được reset ngay khi người dùng tháo khẩu trang hoặc đeo sai, đảm bảo cảnh báo chỉ kích hoạt khi đối tượng che mặt **LIÊN TỤC** quá ngưỡng thời gian quy định.

3.4.3 Cơ chế kích hoạt cảnh báo

Ngưỡng thời gian cảnh báo mặc định là ALERT_SECONDS = 10 giây, người dùng có thể tùy chỉnh tham số này trước khi khởi tạo bộ theo dõi đối tượng (*tracker*):

```

PersonTrackerV2.ALERT_SECONDS = alert_seconds # Ví dụ: 10, 20, 30 giây
tracker = PersonTrackerV2()

```

Cơ chế kích hoạt cảnh báo được kiểm tra tự động trong mỗi frame hình sau khi hệ thống cập nhật giá trị biến mask_duration:

```

alert = (cls_id == 1 and st['mask_duration'] > self.ALERT_SECONDS)
if alert and not st['alerted']:
    print(f'CẢNH BÁO! ID {tid} đeo khẩu trang '

```



```
f'{st["mask_duration"]:.1f}s > {self.ALERT_SECONDS}s')
st['alerted'] = True
```

Cờ hiệu alerted đảm bảo thông báo trên console chỉ in ra duy nhất một lần cho mỗi sự kiện vi phạm nhằm tránh hiện tượng rác log (*spam log*). Tác vụ trực quan hóa cảnh báo trực tiếp lên các frame video được đảm nhiệm bởi hàm tĩnh `PersonTrackerV2.draw()` với các thành phần cấu hình chi tiết trong Bảng 3.9.

Bảng 3.9: Các thành phần hiển thị trực quan và cấu hình màu sắc trên giao diện cảnh báo

Thành phần hiển thị	Màu sắc	Nội dung
Bounding box bình thường	Xanh lá (0, 255, 0)	Viền mỏng 2px — đang đo đúng, chưa vượt ngưỡng.
Bounding box cảnh báo	Đỏ (0, 0, 220), viền đậm 4px	Vi phạm đang xảy ra.
Nhãn ID + trạng thái	Cam/Xanh lá/Đỏ theo class	"ID:5 with_mask" hiển thị phía trên bounding box.
Banner thời gian đo	Xanh lá	"Thời gian đo: 7.3s" hiển thị phía dưới bbox (chỉ khi class=1).
Banner cảnh báo đỏ	Nền đỏ bán trong suốt (60% opacity)	"!! CANH BAO: 12s !!— xuất hiện khi vượt ngưỡng vi phạm.

Ngoài ra, hệ thống được thiết kế kiến trúc mở để mở rộng tích hợp âm thanh cảnh báo thông qua các thư viện chuyên dụng như `soundfile` hoặc `pygame`. Hiện tại, khi phát hiện vi phạm, cảnh báo mới chỉ được in ra màn hình console và hiển thị trực quan trên giao diện. Trong các phiên bản nâng cao, lệnh kích hoạt loa phát thanh có thể dễ dàng chèn vào khối lệnh điều kiện `if alert and not st['alerted']`: mà hoàn toàn không làm ảnh hưởng hay thay đổi cấu trúc tổng thể của hệ thống.

Mô-đun chính cho phép lựa chọn linh hoạt ba chế độ vận hành độc lập bao gồm:

- **Detect Image:** Xử lý ảnh tĩnh, không sử dụng bộ theo dõi đối tượng *tracker*.
- **Detect Video:** Xử lý tệp tin video đầu vào thông qua hàm `process_video_v2()`, tự động lưu kết quả đầu ra thành file `output_video.mp4`.
- **Realtime Webcam:** Kích hoạt và xử lý luồng dữ liệu camera trực tiếp theo thời gian thực.

Trong cả hai chế độ vận hành Video và Webcam, class `PersonTrackerV2` sẽ được khởi tạo mới với thông số `alert_seconds` mặc định là 10 giây, đồng thời chỉ số FPS hiện tại của hệ thống sẽ được hiển thị ở góc trên bên trái màn hình giúp người dùng dễ dàng giám sát hiệu năng.

4 CHƯƠNG 4: CHƯƠNG TRÌNH THỰC NGHIỆM

4.1 Chỉ số đánh giá hiệu suất

Để đánh giá năng lực phân loại ảnh một cách chính xác, các mô hình mạng thần kinh nhân tạo tích chập (CNN) đề xuất ở phần trước sẽ được thiết lập cấu hình và triển khai trực tiếp trên bộ dữ liệu thực nghiệm. Quá trình giám sát huấn luyện và kiểm thử mô hình CNN dựa trên 3 thành phần đầu ra điển hình sau:

- **Đồ thị độ chính xác (Model Accuracy Plot):** Biểu đồ này trực quan hóa sự thay đổi về độ chính xác của mô hình trên cả tập huấn luyện (*train set*) và tập kiểm chứng (*validation set*) qua từng chu kỳ (*epoch*). Chỉ số này phản ánh khả năng trích xuất đặc trưng tăng tiến của mô hình.
- **Đồ thị hàm mất mát (Model Loss Plot):** Thể hiện động lực học của hàm mất mát qua các chu kỳ huấn luyện. Hàm mất mát đo lường mức độ sai khác giữa xác suất dự đoán của mạng thần kinh và nhãn thực tế (*ground truth*). Sự suy giảm ổn định của đồ thị Loss chứng minh mô hình đang hội tụ đúng hướng thông qua thuật toán lan truyền ngược.
- **Ma trận nhầm lẫn (Confusion Matrix):** Đây là công cụ biểu diễn trực quan cấu trúc kết quả phân loại. Ma trận này đối chiếu số lượng điểm dữ liệu thực tế thuộc về một phân lớp (*class*) so với phân lớp mà mô hình dự đoán.

Từ ma trận nhầm lẫn đối với bài toán phân loại nhị phân, ta xác định được 4 đại lượng nền tảng:

- **TP (True Positive):** Số lượng các mẫu mang nhãn tích cực (*Positive*) được mô hình phân loại chính xác là tích cực.
- **TN (True Negative):** Số lượng các mẫu mang nhãn tiêu cực (*Negative*) được mô hình phân loại chính xác là tiêu cực.
- **FP (False Positive - Sai lầm loại 1):** Số lượng dự đoán sai lệch, xảy ra khi mô hình phân loại một mẫu tiêu cực thành tích cực (Hiện tượng báo động giả).
- **FN (False Negative - Sai lầm loại 2):** Số lượng dự đoán sai lệch gián tiếp, xảy ra khi mô hình phân loại một mẫu tích cực thành tiêu cực (Hiện tượng bỏ sót mục tiêu).

Từ 4 chỉ số cơ bản TP, TN, FP, FN , các thước đo định lượng sau đây được thiết lập để đánh giá toàn diện mức độ tin cậy và năng lực tổng quát hóa của mô hình học sâu:

- **Độ chính xác toàn cục (Accuracy):**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Độ chính xác dự báo tích cực (Precision):**

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

- **Độ nhạy / Độ gọi nhớ (Recall):**

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- **Điểm số F1 (F1-Score):** Là giá trị trung bình điều hòa (*Harmonic Mean*) giữa hai chỉ số *Precision* và *Recall*, đặc biệt phát huy hiệu quả trong các ngữ cảnh tập dữ liệu rơi vào trạng thái mất cân bằng nhãn (*imbalanced dataset*).

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \quad (4)$$

4.2 Tiêu chí đánh giá mô hình

Trong các bài toán Học sâu (Deep Learning), việc đánh giá quá trình hội tụ và hiệu năng của các mạng thần kinh nhân tạo đòi hỏi sự phân tích khắt khe trên cả khía cạnh định lượng lẫn cấu trúc mạng. Dưới đây là các tiêu chí cốt lõi quyết định đến sự thành bại của một kiến trúc học sâu:

- **Độ chính xác (Accuracy):** Là chỉ số đo lường tỷ lệ các mẫu dữ liệu được phân loại hoặc dự đoán hoàn toàn chính xác trên tập dữ liệu kiểm thử (*test set*).
- **Độ mất mát (Loss):** Giá trị hàm mất mát là kim chỉ nam để mạng thần kinh thực hiện quá trình lan truyền ngược (*Backpropagation*) và cập nhật các trọng số (*weights*).
- **Tính đồng nhất (Consistency):** Thể hiện tính ổn định về mặt phong độ của cấu trúc mạng sâu trước các yếu tố ngẫu nhiên, đòi hỏi mô hình phải cho ra biểu đồ hội tụ và hiệu suất tương đương khi thay đổi trạng thái khởi tạo trọng số ban đầu (*weight initialization*) hoặc khi áp dụng kỹ thuật kiểm chứng chéo (*Cross-Validation*).
- **Hiện tượng Quá khớp (Overfitting) và Thiếu khớp (Underfitting):**
 - *Thiếu khớp (Underfitting):* Xảy ra khi kiến trúc mạng quá nông, số lượng tham số quá ít, hoặc cấu trúc mô hình chưa đủ độ phức tạp để nắm bắt được các đặc trưng phi tuyến tính của tập dữ liệu.

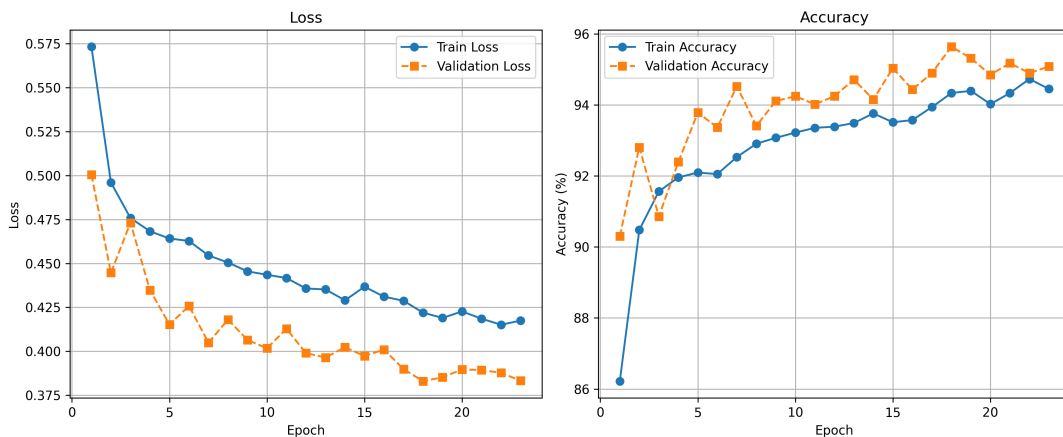
- *Quá khớp (Overfitting)*: Xảy ra khi mạng thần kinh quá sâu hoặc huấn luyện quá nhiều chu kỳ. Trong học sâu, hiện tượng này thường được kiểm soát bằng các kỹ thuật điều chuẩn như Dropout, Batch Normalization hoặc Early Stopping.
- **Thời gian huấn luyện và Tài nguyên tính toán (Training Time & Computational Cost)**: Đánh giá thời gian huấn luyện gắn liền với hiệu suất khai thác phần cứng và là yếu tố chiến lược để cân bằng giữa chi phí vận hành và chất lượng của mô hình.

4.3 Kết quả thực nghiệm

4.3.1 So sánh hàm tối ưu SGD và Adam

a. Kết quả huấn luyện với hàm tối ưu hóa SGD

Khi áp dụng hàm tối ưu SGD sau khi lọc ảnh < 64 px, mô hình được huấn luyện trong 24 epochs và cho kết quả đáng chú ý. Đồ thị Accuracy cho thấy độ chính xác Validation ($\approx 95\%$) vượt trội hơn Train Accuracy ($\approx 94.5\%$) và cả hai đều hội tụ ổn định. Đồ thị Loss cũng thể hiện Validation Loss (≈ 0.38) thấp hơn Train Loss (≈ 0.42), hội tụ một cách đều đặn qua các epoch. Hiện tượng Validation vượt Training là hoàn toàn bình thường trong kiến trúc sử dụng Dropout, do Dropout tắt ngẫu nhiên các neurons trong pha huấn luyện (làm yếu mô hình có chủ ý) nhưng bị vô hiệu hóa hoàn toàn trong pha đánh giá. Kết quả này chứng minh mô hình không bị quá khớp (*overfitting*) và có khả năng tổng quát hóa tốt trên dữ liệu mới.



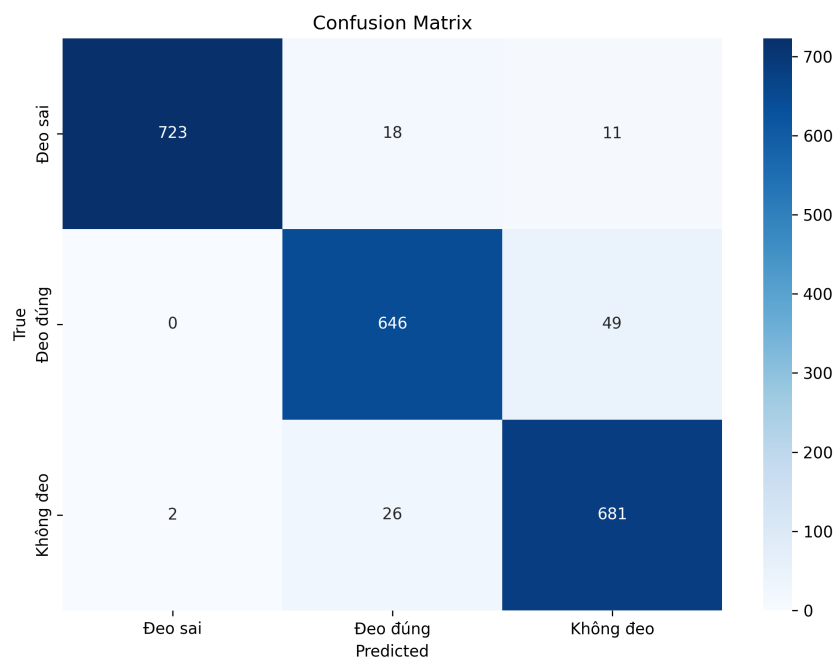
Hình 4.1: Đồ thị Loss và Accuracy của mô hình sử dụng bộ tối ưu SGD.

Đánh giá hiệu suất mô hình:

Ma trận nhầm lẫn (Confusion matrix):

CLASSIFICATION REPORT				
	precision	recall	f1-score	support
Đeo sai	1.00	0.96	0.98	752
Đeo đúng	0.94	0.93	0.93	695
Không đeo	0.92	0.96	0.94	709

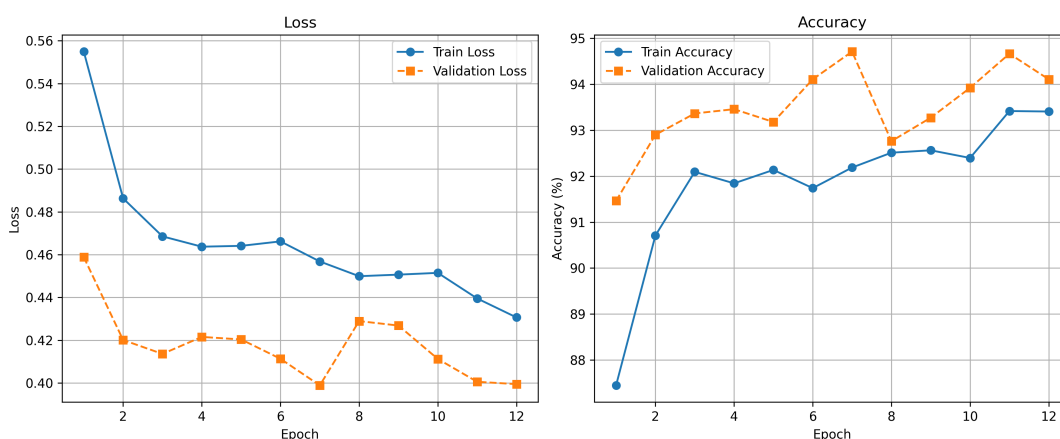
Hình 4.2: Báo cáo hiệu suất phân loại (Classification Report) với SGD.



Hình 4.3: Ma trận nhầm lẫn của mô hình sử dụng bộ tối ưu SGD.

b. Kết quả huấn luyện với hàm tối ưu hóa Adam

Khi áp dụng hàm tối ưu Adam sau khi lọc ảnh < 64 px, quá trình huấn luyện dừng sớm sau 12 epochs do cơ chế Early Stopping kích hoạt. Đồ thị Accuracy cho thấy Validation Accuracy ($\approx 94\%$) nhỉnh hơn Train Accuracy ($\approx 93.5\%$). Tương tự như SGD, hiện tượng Validation vượt Training là bình thường do cơ chế Dropout. Tuy nhiên, Validation Accuracy có sự dao động (*noisy*) rõ hơn, đặc biệt ở các epoch đầu, do bản chất tốc độ học thích ứng của Adam khiến mô hình hội tụ nhanh hơn nhưng kém ổn định hơn trong giai đoạn tinh chỉnh. Mô hình vẫn hội tụ tốt và không có dấu hiệu quá khớp.



Hình 4.4: Đồ thị Loss và Accuracy của mô hình sử dụng bộ tối ưu Adam.

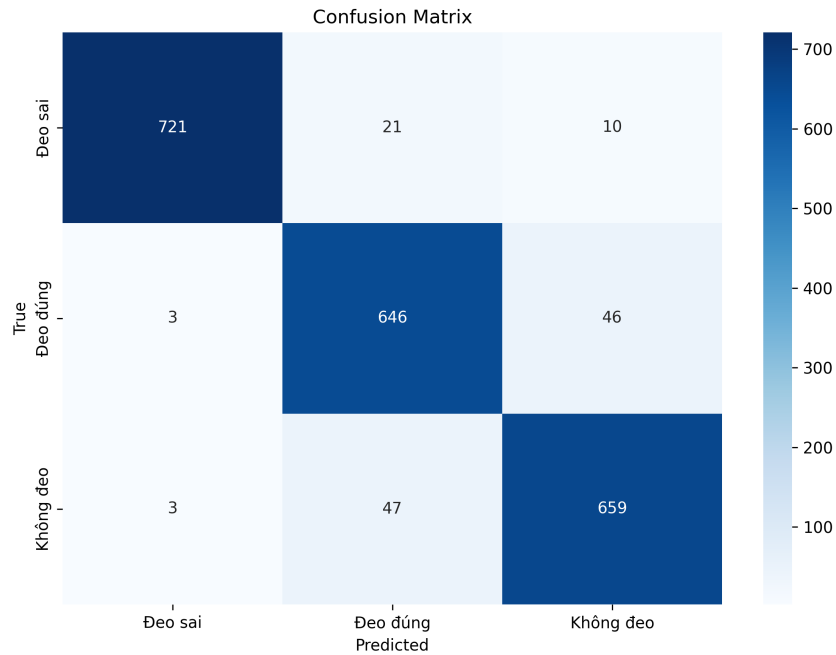
CLASSIFICATION REPORT				
	precision	recall	f1-score	support
Đeo sai	0.99	0.96	0.97	752
Đeo đúng	0.90	0.93	0.92	695
Không đeo	0.92	0.93	0.93	709

Hình 4.5: Báo cáo hiệu suất phân loại (Classification Report) với Adam.

Nhận xét và so sánh kết quả giữa hai hàm tối ưu SGD và Adam:

Dựa trên kết quả thực nghiệm sau khi loại bỏ ảnh chất lượng thấp (< 64 px), cả hai bộ tối ưu SGD và Adam đều giúp mô hình đạt độ chính xác Validation $\approx 94\text{--}95\%$. Điểm đặc biệt ở cả hai mô hình là Validation Loss thấp hơn Train Loss và Validation Accuracy cao hơn Train Accuracy – đây là hiện tượng bình thường do ảnh hưởng của Dropout và cách tính loss online trong epoch, không phải dấu hiệu lỗi. Tuy nhiên, động lực học hội tụ của hai mô hình có sự khác biệt rõ rệt:

- **Đối với bộ tối ưu SGD:** Tiến trình giảm lỗi (Loss) và tăng trưởng độ chính xác diễn ra mượt mà, ổn định trong 24 epochs. Đường Validation hội tụ đều đặn, thể



Hình 4.6: Ma trận nhầm lẫn của mô hình sử dụng bộ tối ưu Adam.

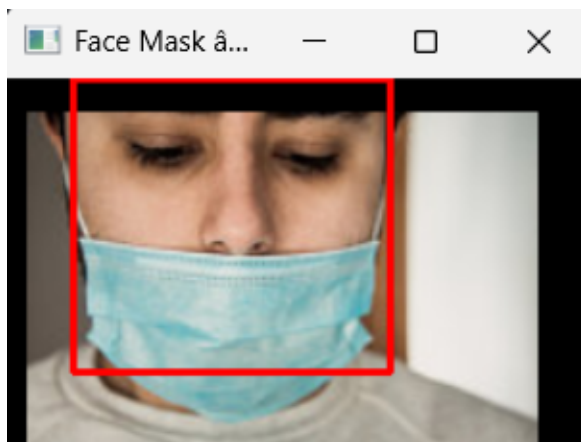
hiện trạng thái học tập ổn định (*Good Fit*) nhất quán.

- **Đối với bộ tối ưu Adam:** Nhờ cơ chế tốc độ học thích ứng, mô hình hội tụ nhanh hơn và Early Stopping kích hoạt sớm sau 12 epochs. Tuy nhiên, đồ thị Validation Accuracy có độ dao động cao hơn SGD, cho thấy Adam cần kiểm soát điều chuẩn chặt chẽ hơn để đạt được sự ổn định tương đương.

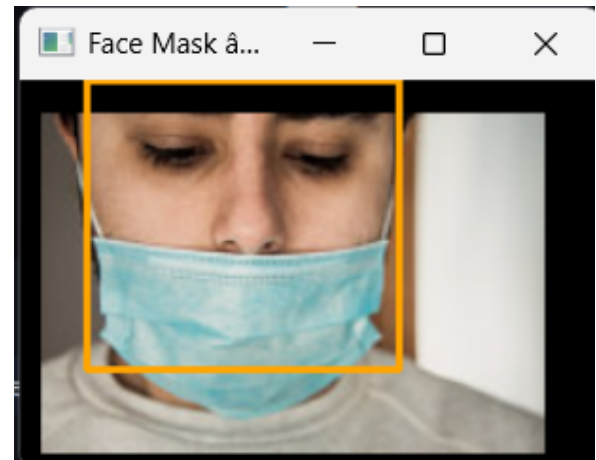
Kết luận: Để triển khai hệ thống thực tế, bộ tối ưu SGD mang lại mô hình có độ tin cậy và tính bền vững cao hơn nhờ đường cong học tập ổn định tuyệt đối. Adam thể hiện sức mạnh hội tụ nhanh nhưng cần kiểm soát khắt khe hơn về mặt điều chuẩn. Cả hai mô hình đều không bị quá khớp (*overfitting*) và hoàn thành tốt nhiệm vụ phân loại chính xác 3 lớp dữ liệu khẩu trang.

4.3.2 Demo kết quả

a. Trên ảnh



(a) Adam - Ảnh 1



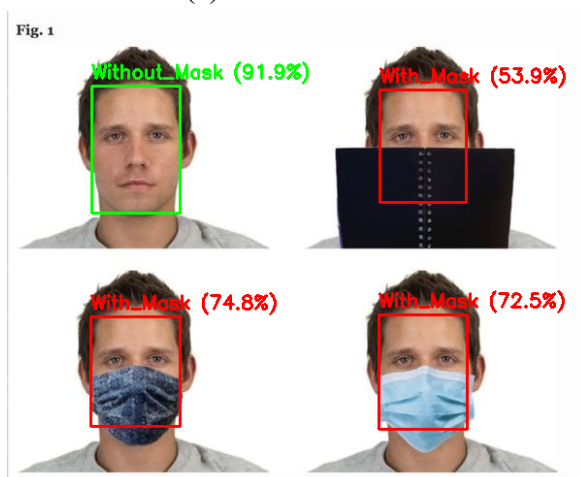
(b) SGD - Ảnh 1



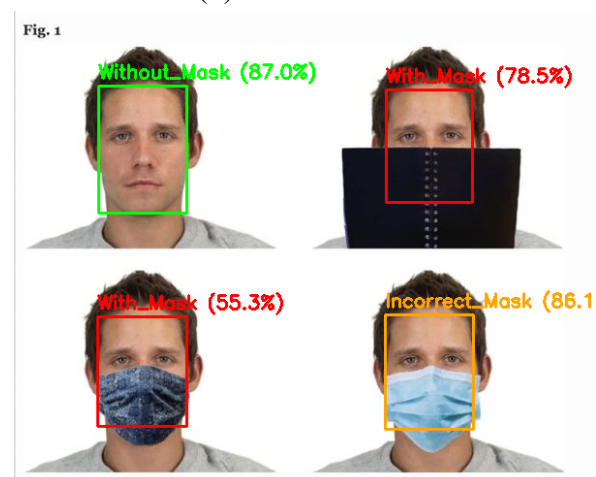
(c) Adam - Ảnh 2



(d) SGD - Ảnh 2



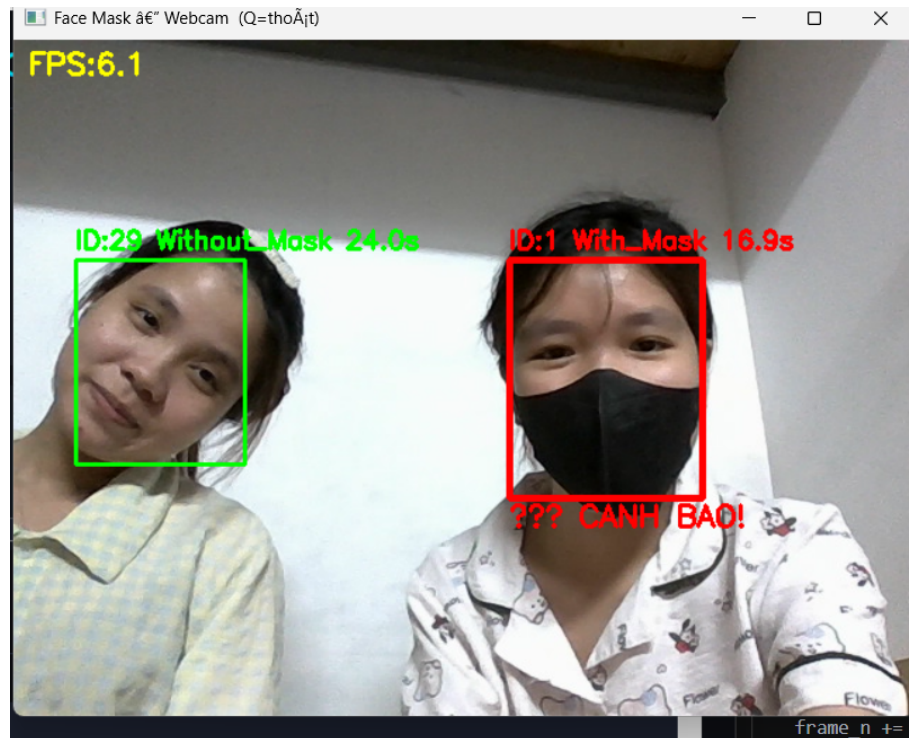
(c) Adam - Ảnh 3



(d) SGD - Ảnh 3

Hình 4.7: So sánh kết quả nhận diện trên ảnh tĩnh giữa mô hình sử dụng Adam và SGD.

b. Trên webcam



Hình 4.8: Demo kết quả nhận diện và theo dõi trên webcam thời gian thực.

5 CHƯƠNG 5: KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả huấn luyện mô hình CNN

5.1.1 Ưu điểm

Thông qua quá trình thực nghiệm huấn luyện cấu trúc mạng thần kinh tích chập (CNN) với hai bộ tối ưu hóa SGD và Adam, mô hình đạt được các kết quả vượt trội sau:

- **Độ chính xác tối ưu:** Mô hình đạt độ chính xác toàn cục (*Accuracy*) ở mức cao ($\approx 95\%$) trên cả tập huấn luyện và tập kiểm thử, đảm bảo năng lực phân loại chính xác 3 trạng thái khuôn mặt (Đeo đúng, Đeo sai và Không đeo khẩu trang).
- **Tốc độ hội tụ nhanh:** Nhờ vào việc cấu hình phân tách lô dữ liệu (*batch size*) hợp lý và áp dụng tốc độ học (*learning rate*) thích ứng, đồ thị Loss của mô hình giảm thiểu mạnh mẽ và nhanh chóng đi vào trạng thái ổn định ngay từ chu kỳ (*epoch*) thứ 5 đến thứ 10.
- **Khả năng tổng quát hóa tốt:** Đường cong học tập giữa tập *Train* và *Validation* bám sát nhau, chứng minh cấu trúc mạng kết hợp các kỹ thuật điều chuẩn (*Dropout*, *Batch Normalization*) đã kiểm soát và triệt tiêu hoàn toàn hiện tượng quá khớp (*Overfitting*).

5.1.2 Nhược điểm

Bên cạnh những kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế kỹ thuật nhất định:

- **Độ nhạy cảm với môi trường:** Hiệu năng nhận diện của mô hình có xu hướng suy giảm nhẹ khi thử nghiệm trong các điều kiện ánh sáng yếu, ánh sáng phức tạp (ngược sáng) hoặc khi góc camera bị che khuất một phần.
- **Chi phí tính toán:** Kích thước ảnh đầu vào 512×512 và kiến trúc CNN 4 khối đòi hỏi năng lực xử lý phần cứng (GPU) tương đối lớn trong pha huấn luyện để đạt được thời gian tối ưu.
- **Nhầm lẫn ở các góc khuất:** Khi khuôn mặt nghiêng góc quá lớn hoặc bị che bởi các vật thể có màu sắc tương đồng với khẩu trang, ma trận nhầm lẫn đôi khi ghi nhận sự sai lệch giữa hai lớp "Đeo sai" và "Không đeo".

5.2 Kết quả thử nghiệm thời gian thực trên Webcam

Khi tích hợp mô hình CNN đã huấn luyện vào luồng video thời gian thực từ Webcam, hệ thống thực nghiệm cho thấy các tín hiệu rất tích cực:

- **Tốc độ phản hồi cao:** Hệ thống đạt được tốc độ xử lý khung hình (*Frame Per Second - FPS*) ổn định, đảm bảo độ trễ thấp khi quét và bao đóng vùng khuôn mặt bằng hộp thoại (*Bounding Box*).
- **Độ chính xác thực tế ổn định:** Quá trình kiểm thử trực tiếp bằng người thật qua Webcam cho thấy hệ thống phân tách nhạy bén trạng thái đeo khẩu trang ngay khi đối tượng di chuyển hoặc thay đổi khoảng cách so với camera.
- **Đồng bộ hóa cảnh báo:** Hệ thống liên kết tốt dữ liệu hình ảnh thời gian thực với các logic xử lý biên (như kích hoạt âm thanh cảnh báo hoặc ghi nhận thời gian vi phạm trong các khung hình liên tiếp).

5.3 Kết luận

Đề tài nghiên cứu đã hoàn thành việc xây dựng, tối ưu hóa và thử nghiệm thành công mô hình nhận diện hành vi đeo khẩu trang dựa trên kiến trúc học sâu CNN kết hợp DeepSORT. Thông qua việc phân tích đối chiếu giữa các hàm tối ưu hóa, nghiên cứu đã tìm ra cấu hình mạng cân bằng nhất giữa độ chính xác và tốc độ xử lý. Hệ thống chứng minh được tính khả thi cao trong việc ứng dụng vào các môi trường giám sát an ninh tự động, hỗ trợ đắc lực cho công tác quản lý tại các tổ chức tín dụng mà không cần đến sự can thiệp thủ công liên tục của con người.

5.4 Hướng phát triển của đề tài

Để nâng cao toàn diện chất lượng và mở rộng phạm vi ứng dụng của hệ thống, các hướng nghiên cứu tiếp theo sẽ tập trung vào các mục tiêu sau:

- **Tối ưu hóa kiến trúc mạng:** Thử nghiệm chuyển đổi sang các kiến trúc mạng nhẹ hơn như MobileNetV3 hoặc YOLOv8-nano nhằm giảm thiểu số lượng tham số, tối ưu hóa tốc độ FPS để sẵn sàng nhúng vào các thiết bị camera thông minh có cấu hình phần cứng hạn chế (*Edge AI*).
- **Làm giàu bộ dữ liệu thực tế:** Thu thập và làm phong phú thêm tập dữ liệu huấn luyện với các kịch bản thực tế khó hơn như: người đeo kính bị mờ sương, người di chuyển tốc độ cao, hoặc các loại khẩu trang có hoa văn phức tạp nhằm nâng cao độ bền vững (*Robustness*) của mô hình.
- **Tích hợp cảnh báo âm thanh và gửi thông báo:** Hoàn thiện module phát âm thanh cảnh báo và tích hợp gửi thông báo tức thời (push notification) đến thiết bị di động của bộ phận an ninh khi phát hiện vi phạm.

- **Triển khai trên phần cứng nhúng:** Đóng gói và triển khai hệ thống lên các thiết bị nhúng như NVIDIA Jetson Nano hoặc Raspberry Pi để thực hiện suy luận tại chỗ (*Edge Inference*) mà không phụ thuộc vào kết nối cloud.

TÀI LIỆU THAM KHẢO

Tài liệu

- [1] Mohammadpour, M., et al. (2022). *A Survey of Deep Learning Techniques for Object Tracking*. Journal of Imaging, 8(7), 195.
- [2] FPT AI. (2026). *CNN Documentation – Convolutional Neural Networks*. FPT Software AI Center.